

Project: Architectural Fingerprinting (Building ID Generation)

Goal: Develop a feature extraction module that converts a 2D image of a building into a unique, structured "ID" (fingerprint) based on its physical and architectural characteristics. This ID must be robust enough to allow the system to recognize and match the same building when viewed from different angles, distances, or lighting conditions in future images.

Input: An image or cropped frame of a single building.

Outcome: A structured metadata object (e.g., a JSON file) representing the unique "Building ID." The ID can include:

- **Floor Count:** The total number of detectable horizontal levels.
- **Window Grid Analysis:** A mapping of the number of windows per floor (e.g., Floor 1: 8 windows, Floor 2: 8 windows).
- **Entrance Mapping:** Count and relative positioning of visible doors or main entry points.
- **Structural Aspect Ratio:** The ratio of the building's visible width to its height, calculated as $R = w/h$.
- **Keypoint Signature:** A set of descriptors derived from permanent structural corners and roofline edges.

And everything else you think can be good as an ID element.

Mentor Tip: Focus on **Semantic Feature Extraction** to handle different image angles, don't rely on raw pixel values. Look at object detection, horizontal dividers (e.g. **Hough Line Transforms**).

The final "ID" should be a "fuzzy match" signature, meaning it should be able to tolerate slight variations or minor occlusions (like a tree partially blocking a window) while still identifying the building correctly.

Project: Eagle-Eye (Image difference)

Goal: Develop a 2D image processing pipeline that aligns two multi-temporal images and identifies minute physical or object-level changes (Added, Removed, Moved) while ignoring lighting, shadows, and weather variations.

Module 1: Robust Image Registration (The "Lock")

Objective: Align the "New" image to the "Reference" image so they share the exact same coordinate system.

- **Input:** Two 2D images (Reference & New) with varying zoom and angles.
- **Process:**
 - **Feature Extraction:** Use **SIFT** or **ORB** to find keypoints.
 - **Homography Mapping:** Compute the transformation matrix to warp the New image.
- **Outcome:** A pixel-perfect overlay where the only differences are actual physical changes, not camera movement.

Module 2: Illumination Invariant Pre-processing (The "Filter")

Objective: Strip away the "lighting" layer of the image to look only at the "structure" or "reflectance" layer.

- **Input:** The aligned image pair.
- **Process:**
 - **Color Space Conversion:** Experiment with **LAB** or **HSV** color spaces where "Luminance" is separated from "Color."
 - **Normalization:** Apply **Histogram Equalization** or **Shadow Removal** algorithms.
 - **Gradient Analysis:** Instead of comparing pixel colors, compare **Image Gradients** (the edges). Shadows change color, but the edge of a pit or a vehicle stays the same.
- **Outcome:** A "Cleaned" version of both images where shadows and brightness are neutralized.
- **Mentor Tip:** Look into **Local Binary Patterns (LBP)**. LBP is excellent for texture-based change detection because it is naturally invariant to monotonic grayscale transformations (lighting changes).

Module 3: Dual-Path Change Detection (The "Sensor")

Objective: Differentiate between "Ground" (texture) changes and "Object" (discrete) changes.

- **Input:** Normalized image pair.
 - **Process:**
 - **Path A (Ground):** Use a **Sliding Window** or **Patch-based Comparison** to find texture anomalies (stains, holes, paths).
 - **Path B (Objects):** Run a small-object detector (like **YOLOv8-Small**) to find vehicles and equipment.
 - **Outcome:** A list of bounding boxes for every potential change area.
-

Module 4: Semantic Logic & Output (The "Reporter")

Objective: Interpret the differences and produce the final visual report.

- **Input:** The detections from Module 3.
- **Logic:**
 - **Added:** Detected in New, but 0% overlap in Reference.
 - **Removed:** Detected in Reference, but 0% overlap in New.
 - **Moved:** High similarity score between an object in Ref and an object in New, but at a different coordinate.
- **Outcome:** A high-res image with a legend:
 -  **Blue Circle:** Small Ground Change (Hole/Stain).
 -  **Green Box:** Added Object.
 -  **Red Box:** Missing Object.
 -  **Yellow Arrow:** Moved Object.

Success Criteria:

1. **Stability:** Does the system work if one image is taken at 10:00 AM (bright) and the other at 4:00 PM (long shadows)?
2. **Sensitivity:** Can the system detect a 50cm x 50cm hole in a 50-meter field?
3. **Accuracy:** Does the system avoid flagging "moving shadows" as "removed objects"?

Data

- Aerial Change Detection in Video Games - [Link](#)
- VisTA Git - [Link](#)